

Homework 3: Markov chains and hidden Markov models

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyana Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

(6) Identify the most likely coding versus non-coding composition of a given DNA sequence

(6a) The Viterbi quantities for sequence TAAC

All probabilities mentioned are $\log_{10}$ -transformed.

The initial state probabilities (I), transition probabilities (T) and emission probabilities (E), that will be used to calculate the Viterbi quantities, $v_k(n)$, are shown below.

Initial state probabilities (I)	C-Region	NC-Region
	-0.4	-0.2

Transition probabilities (T)	C-Region	NC-Region
C-Region	-0.50	-0.15
NC-Region	-0.20	-0.40

Emission probabilities (E)	A	C	G	T
C-Region	-0.7	-0.5	-1.0	-0.4
NC-Region	-0.7	-0.2	-1.0	-1.0

For T at position 1, we should compute $P_{C-Region}(T, 1)$ and $P_{NC-Region}(T, 1)$ or in another notation $v_{C-Region}(1)$ and $v_{NC-Region}(1)$ for T observed.

$$P_{C-Region}(T, 1) = I_{C-Region} + E_{C-Region, T} = -0.4 - 0.4 = -0.8$$

$$P_{NC-Region}(T, 1) = I_{NC-Region} + E_{NC-Region, T} = -0.2 - 1.0 = -1.2$$

For A at position 2, we should compute $P_{C-Region}(A, 2)$ and $P_{NC-Region}(A, 2)$ or in another notation $v_{C-Region}(2)$ and $v_{NC-Region}(2)$ for A observed.

$$\begin{aligned} P_{C-Region}(A, 2) &= E_{C-Region, A} + \max \begin{cases} P_{C-Region}(T, 1) + T_{C-Region, C-Region} \\ P_{NC-Region}(T, 1) + T_{NC-Region, C-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -0.8 - 0.5 \\ -1.2 - 0.2 \end{cases} = -0.7 + \max \begin{cases} -1.3 \\ -1.4 \end{cases} = -2.0 \end{aligned}$$

$$\begin{aligned} P_{NC-Region}(A, 2) &= E_{NC-Region, A} + \max \begin{cases} P_{NC-Region}(T, 1) + T_{NC-Region, NC-Region} \\ P_{C-Region}(T, 1) + T_{C-Region, NC-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -1.2 - 0.4 \\ -0.8 - 0.15 \end{cases} = -0.7 + \max \begin{cases} -1.60 \\ -0.95 \end{cases} = -1.65 \end{aligned}$$

$P_{C-Region}(A, 2)$ comes from position 1 C-region. $P_{NC-Region}(A, 2)$ comes from position 1 C-region.

For A at position 3, we should compute $P_{C-Region}(A, 3)$ and $P_{NC-Region}(A, 3)$ or in another notation $v_{C-Region}(3)$ and $v_{NC-Region}(3)$ for A observed.

$$\begin{aligned} P_{C-Region}(A, 3) &= E_{C-Region, A} + \max \begin{cases} P_{C-Region}(A, 2) + T_{C-Region, C-Region} \\ P_{NC-Region}(A, 2) + T_{NC-Region, C-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -2.0 - 0.5 \\ -1.65 - 0.2 \end{cases} = -0.7 + \max \begin{cases} -2.50 \\ -1.85 \end{cases} = -2.55 \end{aligned}$$

$$P_{NC-Region}(A, 3) = E_{NC-Region, A} + \max \begin{cases} P_{NC-Region}(A, 2) + T_{NC-Region, NC-Region} \\ P_{C-Region}(A, 2) + T_{C-Region, NC-Region} \end{cases}$$

$$= -0.7 + \max \begin{Bmatrix} -1.65 - 0.4 \\ -2.0 - 0.15 \end{Bmatrix} = -0.7 + \max \begin{Bmatrix} -2.05 \\ -2.15 \end{Bmatrix} = -2.75$$

$P_{C-Region}(A, 3)$ comes from position 2 NC-region. $P_{NC-Region}(A, 3)$ comes from position 2 NC-region.

For C at position 4, we should compute $P_{C-Region}(C, 4)$ and $P_{NC-Region}(C, 4)$ or in another notation $v_{C-Region}(4)$ and $v_{NC-Region}(4)$ for C observed.

$$P_{C-Region}(C, 4) = E_{C-Region, C} + \max \begin{Bmatrix} P_{C-Region}(A, 3) + T_{C-Region, C-Region} \\ P_{NC-Region}(A, 3) + T_{NC-Region, C-Region} \end{Bmatrix}$$

$$= -0.5 + \max \begin{Bmatrix} -2.55 - 0.5 \\ -2.75 - 0.2 \end{Bmatrix} = -0.5 + \max \begin{Bmatrix} -3.05 \\ -2.95 \end{Bmatrix} = -3.45$$

$$P_{NC-Region}(C, 4) = E_{NC-Region, A} + \max \begin{Bmatrix} P_{NC-Region}(A, 3) + T_{NC-Region, NC-Region} \\ P_{C-Region}(A, 3) + T_{C-Region, NC-Region} \end{Bmatrix}$$

$$= -0.2 + \max \begin{Bmatrix} -2.75 - 0.4 \\ -2.55 - 0.15 \end{Bmatrix} = -0.2 + \max \begin{Bmatrix} -3.15 \\ -2.70 \end{Bmatrix} = -2.9$$

$P_{C-Region}(C, 4)$ comes from position 3 NC-region. $P_{NC-Region}(C, 4)$ comes from position 3 C-region.

The resulting Viterbi quantities, ($\log_{10}$ -transformed probabilities) are summarised in the table below.

	T	A	A	C
C-Region	-0.80	-2.00	-2.55	-3.45
NC-Region	-1.20	-1.65	-2.75	-2.90

The pointers (previous state visited) are summarised in the table below.

Pointers	T	A	A	C
C-Region	Not applicable	Position 1, C region	Position 2, NC region	Position 3, NC region

Pointers	T	A	A	C
NC-Region	Not applicable	Position 1, C region	Position 2, NC region	Position 3, C region

(6b) The most likely hidden path

The state in the last position, where C is observed, with the highest probability is NC-Region, with a probability of -2.9. Using the pointers summarised in the previous tables, the resulting shortest path is: C, NC, C, NC, or more precisely summarised in table format below:

Shortest path	Position 1	Position 2	Position 3	Position 4
Hidden variable state	C-Region	NC-Region	C-Region	NC-Region

(7) Predicting protein secondary structure using HMMs

(7a) Reading and storing data

Saving the tab-delimited files as dataframes

```
proteins_train <- read.delim("exercise/data/proteins_train.tsv", sep="\t", header = FALSE)
proteins_test <- read.delim("exercise/data/proteins_test.tsv", sep="\t", header = FALSE)
proteins_new <- read.delim("exercise/data/proteins_new.tsv", sep="\t", header = FALSE)
```

Renaming the columns.

```
colnames(proteins_train) <- c("Protein ID", "Amino acid sequence", "Secondary structure code")
colnames(proteins_test) <- c("Protein ID", "Amino acid sequence", "Secondary structure code")
colnames(proteins_new) <- c("Protein ID", "Amino acid sequence")
```

(7b) Initial state probabilities (I), transition probabilities (T) and emission probabilities (E)

```
L <- nrow(proteins_train)

aa_codes <- c("A", "C", "D", "E", "F", "G", "H", "I", "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W")

ss_codes <- c("B", "C", "E", "G", "H", "I", "S", "T")
```

```

I_counts <- matrix(0, nrow = length(ss_codes), ncol = 1, dimnames = list(ss_codes))
T_counts <- matrix(0, nrow = length(ss_codes), ncol = length(ss_codes), dimnames = list(ss_codes, ss_codes))
E_counts <- matrix(0, nrow = length(ss_codes), ncol = length(aa_codes), dimnames = list(ss_codes, aa_codes))

for (protein in 1:L) {

  aa_seq <- proteins_train$`Amino acid sequence`[protein]
  ss_seq <- proteins_train$`Secondary structure code`[protein]

  # initialization counts, C is the secondary structure of the first amino acid
  first_ss <- substr(ss_seq, 1, 1)
  I_counts[first_ss,] <- I_counts[first_ss,] + 1

  l_seq = nchar(aa_seq)

  for (N in 1:l_seq) {
    aa_N <- substr(aa_seq, N, N)
    ss_N <- substr(ss_seq, N, N)

    # emission counts
    E_counts[ss_N, aa_N] <- E_counts[ss_N, aa_N] + 1

    # transmission counts
    if (N != 1) {
      T_counts[prev_ss_N, ss_N] <- T_counts[prev_ss_N, ss_N] + 1
    }
    prev_ss_N <- ss_N
  }
}

# maximum likelihood estimates
I <- I_counts/sum(I_counts)
E <- E_counts/rowSums(E_counts)
T <- T_counts/rowSums(T_counts)

print(I)

```

```

[,1]
B    0
C    1
E    0

```

```
G    0
H    0
I    0
S    0
T    0
```

```
print(T)
```

```

      B      C      E      G      H      I
B 0.0185089974 0.60874036 0.0303341902 0.020565553 0.02365039 0.000000e+00
C 0.0288740867 0.51503859 0.1113882450 0.025176795 0.08471493 0.000000e+00
E 0.0039195491 0.10814522 0.8126054988 0.004520356 0.00557892 0.000000e+00
G 0.0078616352 0.11198463 0.0186932215 0.697938505 0.03336827 0.000000e+00
H 0.0004412595 0.01782689 0.0003000565 0.002859362 0.91049492 1.765038e-05
I 0.0000000000 0.00000000 0.0000000000 0.000000000 0.05882353 8.235294e-01
S 0.0258777546 0.38250556 0.0857874520 0.018262686 0.06738999 6.738999e-05
T 0.0179059669 0.22709484 0.0695866617 0.012866563 0.04026162 5.361068e-05

      S      T
B 0.15372751 0.14447301
C 0.13873647 0.09607089
E 0.02932509 0.03590536
G 0.05765199 0.07250175
H 0.01623835 0.05182152
I 0.00000000 0.11764706
S 0.35656042 0.06354876
T 0.12040959 0.51182115
```

```
print(E)
```

```

      A      C      D      E      F      G      H
B 0.04524422 0.03239075 0.06580977 0.02467866 0.04215938 0.05192802 0.03239075
C 0.06450873 0.01585481 0.07227845 0.04888328 0.03388859 0.08271453 0.02583216
E 0.05804938 0.02348868 0.02798043 0.04002518 0.05247046 0.04809315 0.02752267
G 0.10814116 0.01677149 0.09696017 0.08333333 0.03511530 0.06289308 0.03022362
H 0.12500000 0.01249647 0.05261579 0.08629271 0.03646569 0.03503601 0.02188647
I 0.23529412 0.00000000 0.17647059 0.00000000 0.00000000 0.00000000 0.00000000
S 0.05445111 0.01536492 0.08605701 0.05215985 0.02331694 0.14219287 0.02304738
T 0.06342143 0.01168713 0.07736021 0.06015118 0.02348148 0.19911006 0.02122983

      I      K      L      M      N      P      Q
B 0.06221080 0.04575835 0.07506427 0.01645244 0.05449871 0.04215938 0.02982005
C 0.04323518 0.06118295 0.07018550 0.02319447 0.05708306 0.08277187 0.03145159
```

E	0.09306783	0.05767745	0.10511258	0.02374617	0.02400366	0.01782393	0.03135639
G	0.03162124	0.05957372	0.05835080	0.01502446	0.05241090	0.06446541	0.03546471
H	0.05491034	0.07296668	0.11285654	0.02974089	0.03641274	0.01929187	0.04131954
I	0.05882353	0.00000000	0.23529412	0.05882353	0.00000000	0.00000000	0.00000000
S	0.02749511	0.07244423	0.05067727	0.01489319	0.05492284	0.05761844	0.03093200
T	0.02466091	0.07918297	0.04975071	0.01302740	0.06733501	0.06792473	0.03683054
	R	S	T	U	V	W	
B	0.04627249	0.04627249	0.08020566	0.000000000	0.13110540	0.008740360	
C	0.04733507	0.07981880	0.07184839	0.000000000	0.05106224	0.008171106	
E	0.03833720	0.05198409	0.07779018	0.000000000	0.12651275	0.019740795	
G	0.03913347	0.06970650	0.04297694	0.000174703	0.02428372	0.024633124	
H	0.06412384	0.04048997	0.04356114	0.000000000	0.07183705	0.013061282	
I	0.11764706	0.00000000	0.00000000	0.000000000	0.05882353	0.058823529	
S	0.05364243	0.08807871	0.07224206	0.000000000	0.03935575	0.016106207	
T	0.04122661	0.06438643	0.04444325	0.000000000	0.02021123	0.010132418	
	X	Y					
B	0.0010282776	0.06580977					
C	0.0009461280	0.02775309					
E	0.0000000000	0.05521701					
G	0.0000000000	0.04874214					
H	0.0000000000	0.02963499					
I	0.0000000000	0.00000000					
S	0.0002695599	0.02473212					
T	0.0001072214	0.02433925					

(7c) The stationary distribution π

An ergodic Markov chain has a unique stationary distribution $\pi = (\pi_l)_{l \in [K]}$ such that

$$\lim_{n \rightarrow \infty} T_{kl}^n = \pi_l = \sum_{k \in [K]} \pi_k T_{kl}, \quad l \in [K], \quad \sum_{l \in [K]} \pi_l = 1$$

independent of the initial distribution I . In our situation:

$$\sum_{k=1}^8 \pi_k = 1$$

In matrix notation, π is the solution of $\pi^t = \pi^t T$. π is the eigenvector of the transpose of T corresponding to an eigenvalue of 1.

```

decomposed_T <- eigen(t(T))
idx_value1 <- which.min(abs(decomposed_T$values - 1))
vector_value1 <- decomposed_T$vectors[,idx_value1]

pi <- vector_value1/sum(vector_value1)
pi <- Re(pi)
pi <- pi / sum(pi)
names(pi) <- ss_codes

pi_matrix = matrix(pi, dimnames = list(ss_codes))
print((pi_matrix))

```

```

      [,1]
B 0.0116560594
C 0.2042297412
E 0.2094674769
G 0.0343029736
H 0.3395299222
I 0.0001018782
S 0.0889276425
T 0.1117843060

```

```

pi_next <- as.numeric(pi %*% T)
names(pi_next) <- ss_codes

print(all.equal(pi, pi_next))

```

```
[1] TRUE
```

```
print(cbind(Original_pi = pi, After_One_Step = pi_next))
```

	Original_pi	After_One_Step
B	0.0116560594	0.0116560594
C	0.2042297412	0.2042297412
E	0.2094674769	0.2094674769
G	0.0343029736	0.0343029736
H	0.3395299222	0.3395299222
I	0.0001018782	0.0001018782
S	0.0889276425	0.0889276425
T	0.1117843060	0.1117843060

(7d) The latent state sequence Z using Viterbi algorithm

```
source("exercise/code/viterbi.r")
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
log_I <- log(I)
log_T <- log(T)
log_E <- log(E)

# for proteins_test
proteins_test_pred <- proteins_test
colnames(proteins_test_pred) <- c("ProteinID", "AminoAcids", "SecondaryStructure")

proteins_test_result <- viterbi(
  E = log_E,
  Tr = log_T,
  I = log_I,
  p = proteins_test_pred
)

# proteins_test_result as tsv file
write.table(proteins_test_result, file="proteins_test_pred.tsv",
            quote=FALSE, sep="\t", col.names = FALSE, row.names = FALSE)

# for proteins_new
proteins_new_pred <- proteins_new
colnames(proteins_new_pred) <- c("ProteinID", "AminoAcids")

proteins_new_result <- viterbi(
  E = log_E,
```

```

Tr = log_T,
I = log_I,
p = proteins_new_pred
)

# protein_new_result as tsv file
write.table(proteins_new_result, file="proteins_new_pred.tsv",
            quote=FALSE, sep="\t", col.names = FALSE, row.names = FALSE)

```

(7e) Confidence intervals via bootstrapping

```

B <- 1000
n_proteins <- nrow(proteins_train)

estimate_params <- function(df, ss_codes, aa_codes) {
  n_rows <- nrow(df)
  I_counts <- matrix(0, nrow = length(ss_codes), ncol = 1, dimnames = list(ss_codes, "Init"))
  T_counts <- matrix(0, nrow = length(ss_codes), ncol = length(ss_codes), dimnames = list(ss_codes, ss_codes))
  E_counts <- matrix(0, nrow = length(ss_codes), ncol = length(aa_codes), dimnames = list(ss_codes, aa_codes))

  for (protein in 1:n_rows) {
    aa_seq <- df$`Amino acid sequence`[protein]
    ss_seq <- df$`Secondary structure code`[protein]

    first_ss <- substr(ss_seq, 1, 1)
    I_counts[first_ss, ] <- I_counts[first_ss, ] + 1

    l_seq <- nchar(aa_seq)
    if (l_seq > 0) {
      aa_split <- strsplit(aa_seq, "")[[1]]
      ss_split <- strsplit(ss_seq, "")[[1]]

      for (N in 1:l_seq) {
        E_counts[ss_split[N], aa_split[N]] <- E_counts[ss_split[N], aa_split[N]] + 1
        if (N > 1) {
          T_counts[ss_split[N - 1], ss_split[N]] <- T_counts[ss_split[N - 1], ss_split[N]] + 1
        }
      }
    }
  }
}

```

```

I_probs <- I_counts / sum(I_counts)
row_sums_E <- rowSums(E_counts)
E_probs <- ifelse(row_sums_E == 0, 0, E_counts / row_sums_E)
row_sums_T <- rowSums(T_counts)
T_probs <- ifelse(row_sums_T == 0, 0, T_counts / row_sums_T)

list(I = I_probs, T = T_probs, E = E_probs)
}

I_boot <- array(0, dim = c(length(ss_codes), 1, B), dimnames = list(ss_codes, "Init", NULL))
T_boot <- array(0, dim = c(length(ss_codes), length(ss_codes), B), dimnames = list(ss_codes,
E_boot <- array(0, dim = c(length(ss_codes), length(aa_codes), B), dimnames = list(ss_codes,

set.seed(42)

for (b in 1:B) {
  # Sample with replacement
  resampled_indices <- sample(1:n_proteins, n_proteins, replace = TRUE)
  boot_data <- proteins_train[resampled_indices, ]

  params <- estimate_params(boot_data, ss_codes, aa_codes)
  I_boot[, , b] <- params$I
  T_boot[, , b] <- params$T
  E_boot[, , b] <- params$E
}

# Compute the empirical 95% confidence intervals (2.5% and 97.5% quantiles)
# We use the apply() function over the 1st and 2nd dimensions (rows and columns)
I_lower <- apply(I_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
I_upper <- apply(I_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

T_lower <- apply(T_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
T_upper <- apply(T_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

E_lower <- apply(E_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
E_upper <- apply(E_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

dimnames(I_lower) <- list(ss_codes, "Init")
dimnames(I_upper) <- list(ss_codes, "Init")
dimnames(T_lower) <- list(ss_codes, ss_codes)
dimnames(T_upper) <- list(ss_codes, ss_codes)
dimnames(E_lower) <- list(ss_codes, aa_codes)

```

```
dimnames(E_upper) <- list(ss_codes, aa_codes)
```

```
# To verify
print(I_lower)
```

```
      Init
B      0
C      1
E      0
G      0
H      0
I      0
S      0
T      0
```

```
print(I_upper)
```

```
      Init
B      0
C      1
E      0
G      0
H      0
I      0
S      0
T      0
```

```
print(T_lower)
```

	B	C	E	G	H	I
B	0.0133168803	0.0133168803	0.0133168803	0.0133168803	0.0133168803	0.0133168803
C	0.0270266272	0.0270266272	0.0270266272	0.0270266272	0.0270266272	0.0270266272
E	0.0033755392	0.0033755392	0.0033755392	0.0033755392	0.0033755392	0.0033755392
G	0.0055771168	0.0055771168	0.0055771168	0.0055771168	0.0055771168	0.0055771168
H	0.0002761404	0.0002761404	0.0002761404	0.0002761404	0.0002761404	0.0002761404
I	0.0000000000	0.0000000000	0.0000000000	0.0000000000	0.0000000000	0.0000000000
S	0.0232727539	0.0232727539	0.0232727539	0.0232727539	0.0232727539	0.0232727539
T	0.0159370286	0.0159370286	0.0159370286	0.0159370286	0.0159370286	0.0159370286

	S	T
B	0.0133168803	0.0133168803

```

C 0.0270266272 0.0270266272
E 0.0033755392 0.0033755392
G 0.0055771168 0.0055771168
H 0.0002761404 0.0002761404
I 0.0000000000 0.0000000000
S 0.0232727539 0.0232727539
T 0.0159370286 0.0159370286

```

```
print(T_upper)
```

```

          B          C          E          G          H          I
B 0.0244670080 0.0244670080 0.0244670080 0.0244670080 0.0244670080 0.0244670080
C 0.0308954167 0.0308954167 0.0308954167 0.0308954167 0.0308954167 0.0308954167
E 0.0044936354 0.0044936354 0.0044936354 0.0044936354 0.0044936354 0.0044936354
G 0.0101883058 0.0101883058 0.0101883058 0.0101883058 0.0101883058 0.0101883058
H 0.0006405149 0.0006405149 0.0006405149 0.0006405149 0.0006405149 0.0006405149
I 0.0000000000 0.0000000000 0.0000000000 0.0000000000 0.0000000000 0.0000000000
S 0.0284622146 0.0284622146 0.0284622146 0.0284622146 0.0284622146 0.0284622146
T 0.0197982674 0.0197982674 0.0197982674 0.0197982674 0.0197982674 0.0197982674
          S          T
B 0.0244670080 0.0244670080
C 0.0308954167 0.0308954167
E 0.0044936354 0.0044936354
G 0.0101883058 0.0101883058
H 0.0006405149 0.0006405149
I 0.0000000000 0.0000000000
S 0.0284622146 0.0284622146
T 0.0197982674 0.0197982674

```

```
print(E_lower)
```

```

          A          C          D          E          F          G          H
B 0.03701587 0.03701587 0.03701587 0.03701587 0.03701587 0.03701587 0.03701587
C 0.06140706 0.06140706 0.06140706 0.06140706 0.06140706 0.06140706 0.06140706
E 0.05535792 0.05535792 0.05535792 0.05535792 0.05535792 0.05535792 0.05535792
G 0.09959287 0.09959287 0.09959287 0.09959287 0.09959287 0.09959287 0.09959287
H 0.12146792 0.12146792 0.12146792 0.12146792 0.12146792 0.12146792 0.12146792
I 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
S 0.05078057 0.05078057 0.05078057 0.05078057 0.05078057 0.05078057 0.05078057
T 0.06021355 0.06021355 0.06021355 0.06021355 0.06021355 0.06021355 0.06021355
          I          K          L          M          N          P          Q

```

B	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587
C	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706
E	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792
G	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287
H	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792
I	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
S	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057
T	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355
	R	S	T	U	V	W	X
B	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587	0.03701587
C	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706	0.06140706
E	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792	0.05535792
G	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287	0.09959287
H	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792	0.12146792
I	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
S	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057	0.05078057
T	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355	0.06021355
	Y						
B	0.03701587						
C	0.06140706						
E	0.05535792						
G	0.09959287						
H	0.12146792						
I	0.00000000						
S	0.05078057						
T	0.06021355						

```
print(E_upper)
```

	A	C	D	E	F	G	H
B	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156
C	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320
E	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927
G	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871
H	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308
I	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000
S	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166
T	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643
	I	K	L	M	N	P	Q
B	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156
C	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320
E	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927

G	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871
H	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308
I	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000
S	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166
T	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643
	R	S	T	U	V	W	X
B	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156	0.05326156
C	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320	0.06758320
E	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927	0.06052927
G	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871	0.11766871
H	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308	0.12847308
I	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000	0.60000000
S	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166	0.05817166
T	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643	0.06722643
	Y						
B	0.05326156						
C	0.06758320						
E	0.06052927						
G	0.11766871						
H	0.12847308						
I	0.60000000						
S	0.05817166						
T	0.06722643						

(7f) Accuracy of predictions

```
compute_accuracy <- function(pred_seq, true_seq) {
  pred_chars <- strsplit(pred_seq, "")[[1]]
  true_chars <- strsplit(true_seq, "")[[1]]
  l_seq <- length(true_chars)

  if (l_seq == 0) {
    return(NA_real_)
  }

  sum(pred_chars == true_chars) / l_seq
}

n_test_proteins <- nrow(proteins_test_result)
accuracies <- numeric(n_test_proteins)
```

```
for (i in 1:n_test_proteins) {  
  real_seq <- proteins_test_result[i, 3]  
  pred_seq <- proteins_test_result[i, 4]  
  accuracies[i] <- compute_accuracy(pred_seq, real_seq)  
}  
  
summary(accuracies)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.007752	0.226253	0.322917	0.319801	0.407240	0.857143

The accuracies vary significantly across the protein sequences, from around 0% (min) to 86 % (maximum). The median and mean of 32% indicate there is room for improvement.